

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Vietnam National University – Ho Chi Minh City · Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering



NỀN TẢNG LẬP TRÌNH CHO PHÂN TÍCH VÀ TRỰC QUAN DỮ LIỆU

Programming Foundations for Data Analysis and Visualization

FINAL ASSIGNMENT REPORT · GROUP G3

Three Data Types, One Workflow

Hotel booking cancellations · Vietnamese news topics · Hanoi temperature forecasts

Lecturer **Dr. Lê Thành Sách**

Group **G3**

Semester 261, academic year 2026–2027

No.	Full name	Student ID	Main contribution
1	Ly Minh Trung	2570349	Tabular and time-series analysis; website design and deployment
2	Dinh Truong Tue Linh	2570441	Text analysis (Vietnamese news classification)
3	Nguyen Hoang Nam	2570261	Text analysis (Vietnamese news classification)

[Project website](#) · [GitHub repository](#) · Videos: to be published

Ho Chi Minh City, October 2026

Contents

1	Group and project overview	3
1.1	Group information	3
1.2	Deliverables	3
1.3	The three datasets	4
1.4	Key results	4
1.5	Shared workflow and tools	4
2	Tabular data: predicting hotel booking cancellations	6
2.1	Problem and objective	6
2.2	Dataset	6
2.3	Exploratory data analysis	8
2.4	Data preparation	10
2.5	Modelling	11
2.6	Results on the test set	12
2.7	Extra experiments	13
2.8	Conclusions	16
3	Text data: Vietnamese news topic classification	18
3.1	Problem and objective	18
3.2	Dataset: collected by the group	18
3.3	Exploratory data analysis	20
3.4	Text preparation	22
3.5	Modelling	23
3.6	Extra experiments	24
3.7	Results on the test set	25
3.8	Conclusions	27
4	Time-series data: forecasting Hanoi's daily temperature	29
4.1	Problem and objective	29
4.2	Dataset	29
4.3	Exploratory data analysis	31



4.4	Data preparation	33
4.5	Modelling	35
4.6	Results on the test set	36
4.7	Extra experiments	38
4.8	Conclusions	40
5	Overall conclusions	42
5.1	What the extra experiments showed	42
5.2	Lessons across the three parts	42
5.3	Limitations and next steps	43
A	Links and reproduction	44

Group and project overview

1.1 Group information

Group **G3** carried out the assignment of the course *Nền tảng lập trình cho phân tích và trực quan dữ liệu* (Programming Foundations for Data Analysis and Visualization), Semester 261, academic year 2026–2027, under the supervision of Dr. Lê Thành Sách at the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (VNU-HCM).

Table 1.1. Members of Group G3 and their main contributions.

No.	Full name	Student ID	Main contribution
1	Ly Minh Trung	2570349	Tabular and time-series analysis; website design and deployment
2	Dinh Truong Tue Linh	2570441	Text analysis (Vietnamese news classification)
3	Nguyen Hoang Nam	2570261	Text analysis (Vietnamese news classification)

As a group of three, we completed the three data types the assignment requires (section 3.2): the mandatory **tabular** and **text** parts, and **time series** as the optional third part. Ly Minh Trung carried out the tabular and time-series parts and designed and deployed the group website; Dinh Truong Tue Linh and Nguyen Hoang Nam carried out the text part.

1.2 Deliverables

Everything is published on the group website (GitHub Pages) at https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/, with the source code in https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment. Each part has its own page that shows the full executed notebook together with its report and video.

Table 1.2. Deliverables of each part (full addresses in Appendix A).

Part	Project page	Notebook (.ipynb)	PDF report	Video
Tabular	page	.ipynb on GitHub	report.pdf	to be published
Text	page	.ipynb on GitHub	report.pdf	to be published
Time series	page	.ipynb on GitHub	report.pdf	to be published

1.3 The three datasets

Table 1.3. Datasets used, one per data type.

Part	Dataset and source	Size	Task
Tabular	Hotel Booking Demand (Data in Brief, 2019), two hotels in Portugal	119,390 bookings × 32 columns	binary classification: will the booking be cancelled?
Text	VnExpress headlines and leads, crawled by the group on 27 September 2026	3,780 articles, 10 sections	multi-class classification: which section does the article belong to?
Time series	Open-Meteo Historical Weather API (ERA5 reanalysis), central Hanoi	7,939 days × 14 variables (2005–2026)	regression: tomorrow’s mean temperature, and 1–7 days ahead

1.4 Key results

Table 1.4. Headline result of each part on its held-out test set.

Part	Task	Dataset	Test metric	Baseline	Ours
Tabular	Will a hotel booking be cancelled? (binary)	Hotel Booking Demand, 119,390 bookings	F1	0.580 (Log. Reg.)	0.708
Text	Which of 10 sections is a news article in?	VnExpress, 3,780 articles (self-crawled)	macro-F1	0.865 (TF-IDF + SVM)	0.892
Time series	Hanoi’s mean temperature tomorrow	Open-Meteo / ERA5, 7,939 days	MAE (°C)	1.081 (persistence)	0.916

1.5 Shared workflow and tools

All three parts follow the same workflow required by the assignment (section 5), and every notebook runs end-to-end on Google Colab (*Run all*) with fixed random seeds, loading its data from the repository.

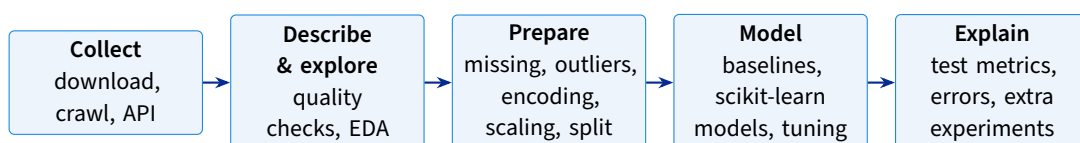


Figure 1.1. The workflow shared by the three parts.

Tools. Python with NumPy and pandas (preparation), Matplotlib and Seaborn (visualisation),

scikit-learn (pipelines, models, tuning, evaluation); requests and BeautifulSoup for crawling, and the sentence-transformers library for the pretrained text encoder. Each notebook also uses object-oriented code: custom scikit-learn transformers (Winsorizer, HotelFeatureBuilder, VietnameseTextCleaner), wrappers (DeltaRegressor) and experiment classes (ModelBenchmark, TextBenchmark, ForecastEvaluator).

How each finding is presented. Every extra experiment answers one explicit question with a figure or table, numbers to compare (before/after, model A vs B, correct vs wrong) and a conclusion in a highlighted box, as asked in section 7.1 of the assignment. All numbers in this report are generated from the notebooks' saved results, so the text always matches the tables.

Tabular data: predicting hotel booking cancellations

2.1 Problem and objective

Hotels take bookings weeks or months in advance, yet 37.0% of the bookings in our dataset were cancelled. A hotel that can tell, at booking time, which reservations are likely to be cancelled can overbook in a controlled way, ask guests to reconfirm, or require a deposit for risky bookings.

Objective. Build a binary classifier that predicts `is_cancelled` (1 = cancelled) using only information that is available when the booking is made, and measure honestly how well it works on bookings it has never seen.

How success is measured. Cancellations are the minority class, so accuracy is misleading: a model that always answers “not cancelled” is already 72.4% accurate on the cleaned data while catching no cancellation at all. We therefore report **F1, recall, ROC-AUC and PR-AUC** (average precision), select models on a validation set, and touch the test set only once.

2.2 Dataset

Hotel Booking Demand [1] contains real bookings from two hotels in Portugal — a *City Hotel* in Lisbon and a *Resort Hotel* in the Algarve — with arrival dates from July 2015 to August 2017 (licence CC BY 4.0). Each row is one booking: 119,390 rows and 32 columns, of which 20 are numerical and 12 categorical.

Table 2.1. Column groups of the dataset.

Group	Columns	Meaning
Target	is_canceled	1 if the booking was cancelled
Timing	lead_time, arrival_date_*	days booked in advance; arrival date
Stay	stays_in_weekend_nights, stays_in_week_nights, adults, children, babies	nights and guests
Channel & guest	market_segment, distribution_channel, customer_type, country, agent, company, is_repeated_guest	booking source, customer type, nationality
History	previous_cancellations, previous_bookings_not_canceled	the guest's past behaviour
Service & price	meal, reserved_room_type, deposit_type, adr, required_car_parking_spaces, total_of_special_requests, booking_changes, days_in_waiting_list	meal plan, room, deposit, average daily rate (ADR)
Leakage	reservation_status, reservation_status_date, assigned_room_type	known only <i>after</i> arrival or cancellation — dropped

Table 2.2. Assignment requirements for tabular data (section 4.2) and how this dataset meets them.

Requirement	This dataset	Status
At least 2,000 samples	119,390 bookings (85,982 after cleaning)	Met
At least 10 columns	32 columns	Met
Missing values	company 94.31%, agent 13.69%, country 0.41%, children; plus hidden Undefined values	Met
Categorical columns (encoding)	12, e.g. hotel, meal, country, market_segment, deposit_type	Met
Numerical columns (scaling)	20, e.g. lead_time, adr, stays_in_week_nights	Met
Harder data (preferred)	imbalance (63.0% / 37.0% raw, 72.4% / 27.6% after cleaning), outliers (ADR up to 5,400), 27.8% duplicates, target leakage	Met

2.3 Exploratory data analysis

2.3.1 Data quality

Table 2.3 lists every quality problem we found and what we did about it. Missing values are not always errors: a missing company means “not booked through a company”, so it becomes a binary flag rather than a deletion. The dataset also hides missing values behind the string `Undefined`.

Table 2.3. Data-quality issues and the action taken for each.

Issue	Affected	Action
Missing company	94.31% of rows	flag <code>has_company</code> (missing = no company)
Missing agent	13.69% of rows	flag <code>has_agent</code> (missing = booked directly)
Missing country, children	0.41% of rows; 4 rows	most-frequent / median imputation, fitted on train
Hidden Undefined	meal 0.98%; market_segment 2 rows; distribution_channel 5 rows	meal: Undefined = SC (per documentation); others → missing → most frequent
Duplicated rows	33,241 (27.8%)	removed before splitting
Invalid bookings	180 with 0 guests; 2 with ADR < 0 or > 1,000	removed
0-night bookings	715 rows	kept — day use is a valid booking
Leakage columns	3 columns	dropped (Section 2.7.1 shows why)

Removing duplicates changes the picture: the cancellation rate falls from 37.0% to 27.6%, because most copies were *cancelled* group bookings. In particular, 93% of the Non Refund bookings were copies (14,587 → 1,010). The clean data is therefore more imbalanced (72.4% / 27.6%).

2.3.2 Cancellation versus key features

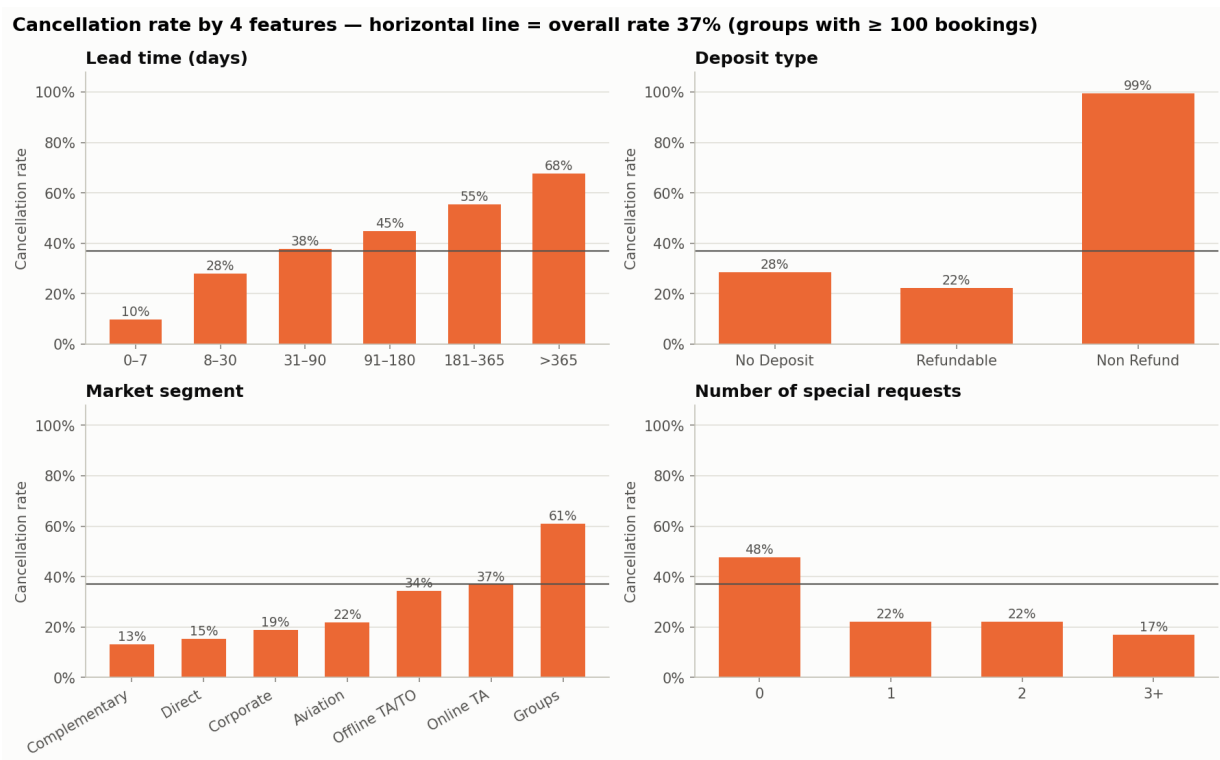


Figure 2.1. Cancellation rate by lead time, deposit type, market segment and number of special requests (raw data, groups with at least 100 bookings). The horizontal line is the overall rate.

Question. Which booking-time attributes are most strongly associated with cancellation?

Finding. **Lead time:** 9.6% of bookings made at most 7 days ahead are cancelled, versus 67.7% of bookings made more than a year ahead. **Non Refund** deposits are cancelled 99.4% of the time (vs 28.4% for no deposit) — counter-intuitive, and largely produced by the duplicated group blocks found above. **Groups** cancel most (61%), **Direct** (15%) and **Corporate** (19%) least. Guests with **special requests** cancel far less (47.7% with none vs 16.8% with three or more).

2.3.3 Distributions, outliers and correlation

The numerical columns are strongly right-skewed and contain thousands of points beyond the IQR fences. Most are real (bookings made two years ahead, expensive suites), so we do not delete them; we clip and transform them instead (Section 2.4). Spearman correlations with the target are only weak to moderate — `lead_time` +0.32, `previous_cancellations` +0.27, `total_of_special_requests` −0.26, `required_car_parking_spaces` −0.20 — so no single column decides the outcome and a multivariate, non-linear model is needed.

2.4 Data preparation

All steps that learn a parameter (medians, clipping quantiles, means and standard deviations, one-hot categories) are fitted on the training set only, inside a scikit-learn Pipeline with a ColumnTransformer. Row-level cleaning learns nothing from the data, so it is done before splitting. The notebook implements the steps as reusable classes: `HotelDataCleaner` (row cleaning with a log), `HotelFeatureBuilder` and `Winsorizer` (custom scikit-learn transformers) and `ModelBenchmark` (trains and scores many models the same way).

Table 2.4. Preparation pipeline.

Step	What is done	Fitted on
Row cleaning	drop leakage columns; remove invalid bookings; remove 33,226 duplicates	— (rules)
Feature engineering	<code>total_nights</code> , <code>total_guests</code> , <code>has_children</code> , <code>is_weekend_only</code> , <code>has_agent</code> , <code>has_company</code> , <code>is_domestic</code> , <code>arrival_weekday</code> ; cyclical <code>month_sin/month_cos</code>	— (stateless)
Missing values	median (numerical), most frequent (categorical)	train
Outliers	Winsorizer: clip to the 0.5%–99.5% quantiles	train
Skewness	<code>log1p</code> on count/day columns; not on <code>adr</code> (1.6% zeros)	—
Encoding	<code>OneHotEncoder</code> , categories below 0.5% merged into “infrequent” → 54 columns	train
Scaling	<code>StandardScaler</code> on the 26 numerical features	train
Split	stratified 70 / 15 / 15: 60,187 / 12,897 / 12,898 bookings	—

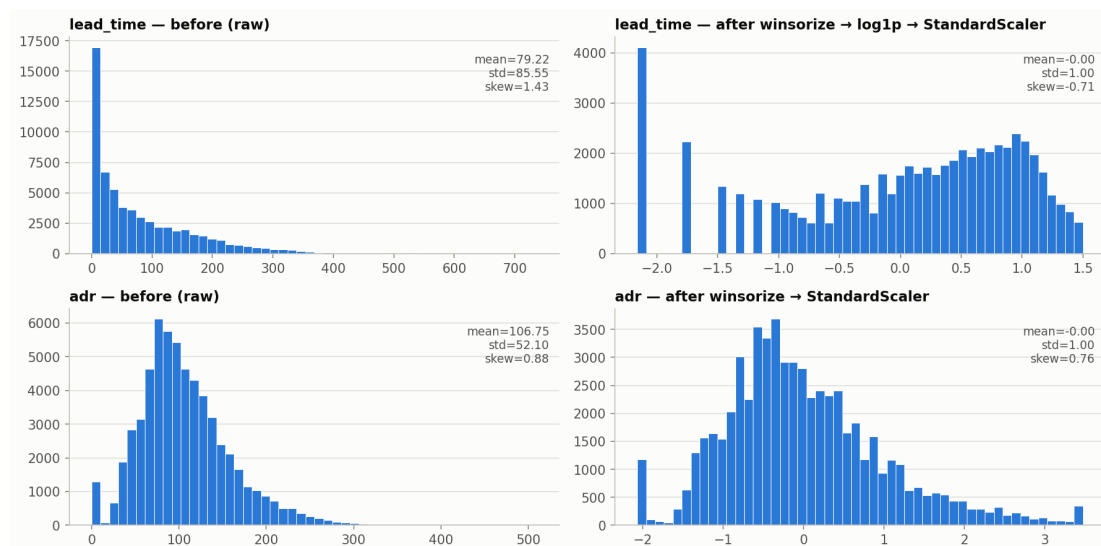


Figure 2.2. Distributions of `lead_time` and `adr` before and after preprocessing (training set).

Question. How does preprocessing change the two most important numerical features?

Finding. Both end with mean 0 and standard deviation 1. The skewness of `lead_time` drops from 1.43 to -0.71 and that of `adr` from 0.88 to 0.76. A first version also applied `log1p` to `adr` and made its skewness *worse* (-3.6) because complimentary rooms cost 0 — which is why `adr` has its own branch.

2.5 Modelling

2.5.1 Model comparison

Five classifiers share the same preprocessing pipeline: a dummy baseline, Logistic Regression, a Decision Tree (depth 12), a Random Forest (200 trees) and HistGradientBoosting (HGB).

Table 2.5. Validation scores (threshold 0.5). The tuned model is described in Section 2.5.2.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Dummy (always “not cancelled”)	0.724	0.000	0.000	0.000	0.500	0.276
Logistic Regression	0.795	0.675	0.493	0.570	0.847	0.684
Decision Tree	0.807	0.669	0.593	0.629	0.858	0.694
Random Forest	0.827	0.743	0.572	0.646	0.888	0.756
HistGradientBoosting (HGB)	0.826	0.723	0.596	0.653	0.887	0.753
HGB, tuned	0.828	0.721	0.613	0.662	0.894	0.768

The dummy model reaches 0.724 accuracy with $F1 = 0$, confirming that accuracy is not informative here. The two ensembles lead and are almost tied — HGB (F1 0.653, PR-AUC 0.753) and Random Forest (F1 0.646, PR-AUC 0.756) — while Logistic Regression is last (F1 0.570, PR-AUC 0.684) despite careful scaling: the signal is non-linear and involves interactions (a long lead time is risky only in some segments).

2.5.2 Hyperparameter tuning

HGB was tuned with `RandomizedSearchCV` (12 configurations $\times$ 3-fold stratified cross-validation on the training set, scored by PR-AUC). The best configuration — learning rate 0.05, 700 boosting iterations, 127 leaf nodes, at least 10 samples per leaf, L2 penalty 0.1 — reached a cross-validated PR-AUC of 0.766 and raised the validation PR-AUC from 0.753 to 0.768 (F1 from 0.653 to 0.662).

2.6 Results on the test set

The final model is the tuned HGB with the decision threshold chosen on the validation set (Section 2.7.2). It was evaluated once on the 12,898 test bookings.

Table 2.6. Test-set results of the baseline and the final model.

Model (test set)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression (baseline)	0.800	0.689	0.501	0.580	0.853	0.694
HGB tuned, threshold 0.33	0.821	0.644	0.786	0.708	0.898	0.776

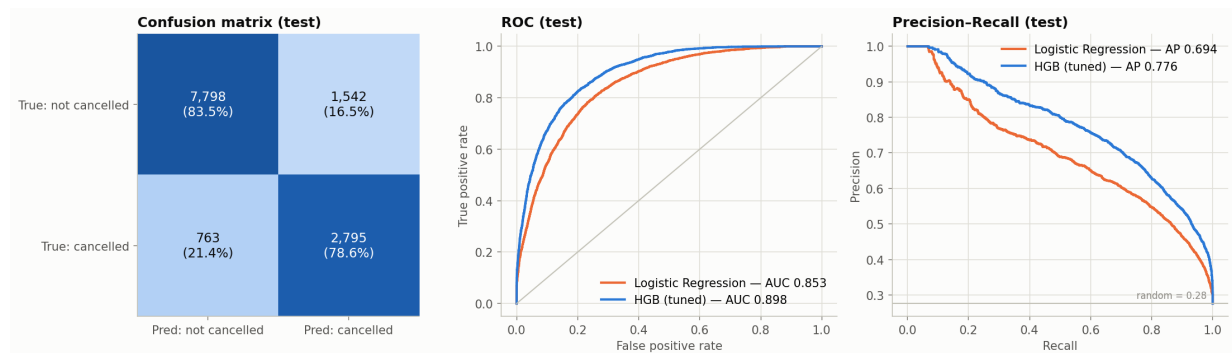


Figure 2.3. Final model on the test set: confusion matrix, ROC curve and precision–recall curve, compared with the Logistic Regression baseline.

Question. How good is the final model on bookings it has never seen?

Finding. F1 rises from 0.580 (baseline) to **0.708** (+0.13), ROC-AUC from 0.853 to **0.898** and PR-AUC from 0.694 to **0.776**. Of the 3,558 real cancellations the model catches 2,795 (recall 0.786); in exchange 1,542 bookings that were kept (16.5%) are flagged by mistake.

2.7 Extra experiments

2.7.1 Two evaluation traps: leakage and duplicates

Table 2.7. Validation scores of the same HGB model under three data-handling scenarios.

Scenario	Accuracy	F1	ROC-AUC	PR-AUC
Leakage columns kept	1.000	1.000	1.000	1.000
Duplicates not removed	0.854	0.793	0.933	0.904
Correct process (this report)	0.826	0.653	0.887	0.753

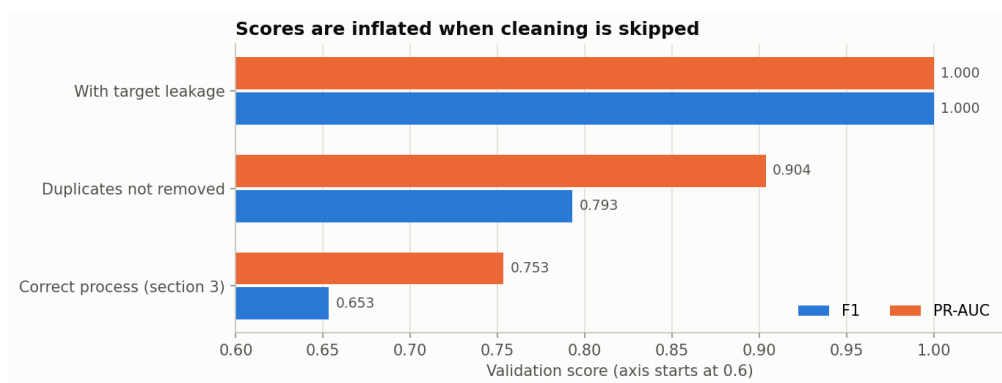


Figure 2.4. F1 and PR-AUC when the cleaning steps are skipped.

Question. If we skipped the cleaning of Section 2.4, how good would the scores look?

Finding. Keeping the leakage columns gives **1.000 on every metric**: the model just reads the answer from `reservation_status`. Keeping duplicates inflates F1 from 0.653 to 0.793 (+0.14) and PR-AUC from 0.753 to 0.904 (+0.15), because copies of the same booking appear in both training and validation. The honest figures are the lower ones.

2.7.2 Class imbalance and the decision threshold

Table 2.8. Three ways to handle the imbalance, on the tuned HGB model (validation set).

Strategy (validation set)	Precision	Recall	F1	PR-AUC
Default threshold 0.5	0.721	0.613	0.662	0.768
<code>class_weight=balanced</code>	0.613	0.814	0.700	0.768
Threshold tuned for F1 (0.33)	0.632	0.784	0.700	0.768

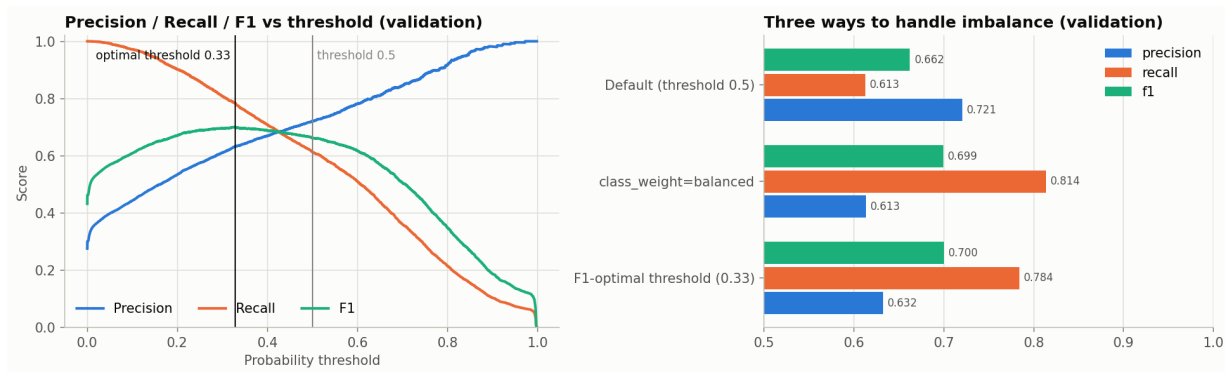


Figure 2.5. Left: precision, recall and F1 as the threshold moves. Right: the three strategies compared.

Question. Does handling the imbalance help, and how?

Finding. At threshold 0.5 the model misses many cancellations (recall 0.613). Re-weighting the classes raises recall to 0.814 and lowering the threshold to 0.33 raises it to 0.784; F1 rises from 0.662 to 0.700 with re-weighting and to 0.700 with the tuned threshold. PR-AUC barely moves (0.768): handling imbalance shifts the operating point along the same curve — trading precision for recall — rather than making the model better at ranking. We keep the tuned threshold because it needs no retraining and can later be set from the hotel’s real costs.

2.7.3 What the model relies on

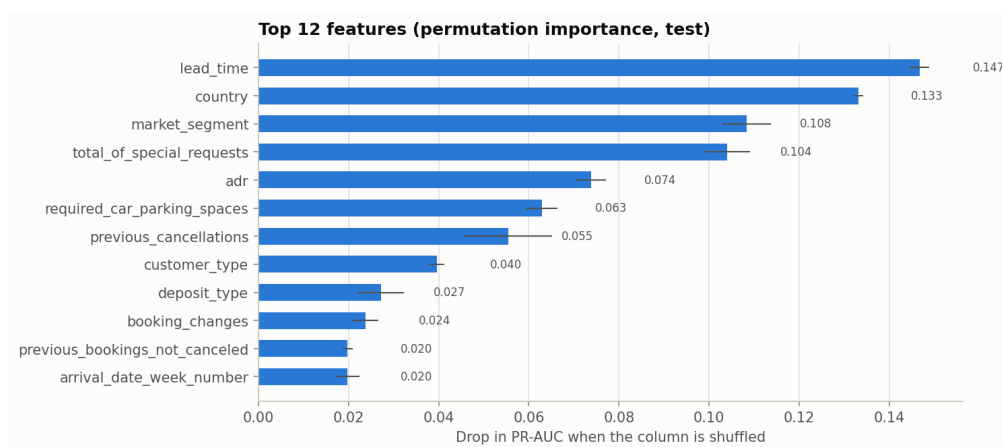


Figure 2.6. Permutation importance on the test set: drop in PR-AUC when one original column is shuffled.

Question. Which columns does the model depend on most?

Finding. lead_time (0.147) and country (0.133) matter most, followed by market_segment (0.108) and total_of_special_requests (0.104) — consistent with Figure 2.1. deposit_type ranks only 9th (0.027): once duplicates are removed, very few Non Refund bookings remain. Permutation importance shows what the model *uses*, not what *causes* a cancellation.

2.7.4 Where the model fails

Table 2.9. Test errors by market segment (segments with at least 50 test bookings).

Market segment	Test bookings	Cancel rate	Error rate	Recall
Online TA	7,693	35.8%	22.6%	0.81
Direct	1,794	13.0%	13.9%	0.41
Complementary	95	11.6%	13.7%	0.09
Corporate	571	12.6%	10.9%	0.42
Offline TA/TO	2,027	14.4%	8.9%	0.87
Groups	686	27.1%	8.5%	0.92

Question. Is the error rate the same for every kind of booking?

Finding. No. Errors concentrate in **Online TA** (error rate 22.6%), the largest segment and the one with the most cancellations (35.8%). For **Direct** and **Corporate** bookings recall is only 0.41 and 0.42: these guests rarely cancel, so the model seldom predicts a cancellation for them. Groups (0.92) and Offline TA/TO (0.87) are caught well. A per-segment threshold would be a natural next step.

2.8 Conclusions

1. **Cleaning decides whether results can be trusted.** 27.8% of the rows were duplicates and three columns leaked the answer; ignoring them gives “excellent” scores (F1 0.793–1.000) that are simply wrong. The true cancellation rate after cleaning is 27.6%, not 37.0%.
2. **Non-linear ensembles work best.** The tuned HistGradientBoosting model beats Logistic Regression by +0.13 F1 and +0.08 PR-AUC on the test set (F1 0.708, ROC-AUC 0.898).
3. **The threshold matters as much as the model.** Moving it from 0.5 to 0.33 raises validation recall from 0.613 to 0.784 at the same ranking quality.
4. **Strongest signals:** booking far ahead, nationality, sales channel, and *not* making special requests.

Limitations. The split is random rather than chronological, so the score on truly future bookings may be lower; the data covers two Portuguese hotels before COVID-19; and the F1-optimal threshold assumes that a missed cancellation and a false alarm cost the same.

Future work. A time-based split (train 2015–2016, test 2017); a threshold derived from real business costs; TargetEncoder for high-cardinality country and agent; probability calibration so scores can drive overbooking directly; per-booking explanations with SHAP.

References

- [1] N. Antonio, A. de Almeida, L. Nunes. *Hotel booking demand datasets*. Data in Brief, 22 (2019), 41–49. <https://doi.org/10.1016/j.dib.2018.11.126>
- [2] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12 (2011), 2825–2830.
- [3] Group G3. *Notebook G3_Tabular_HotelBooking.ipynb*. https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/tabular/G3_Tabular_HotelBooking.ipynb
- [4] Group G3. *Project page (rendered notebook, report and video)*. https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/tabular/ — video: to be published

Text data: Vietnamese news topic classification

3.1 Problem and objective

News websites tag every article with a section so that readers can browse, search and receive recommendations. Given only the **headline** and the **lead paragraph** (“sapo”) of a Vietnamese news article, we predict which of 10 newspaper sections it belongs to: News (Thời sự), World, Business, Sci-Tech, Entertainment, Sports, Law, Education, Health and Travel.

Objective. Build a topic classifier for short Vietnamese texts, compare classical bag-of-words models with a pretrained multilingual sentence-embedding model, and understand which mistakes remain.

How success is measured. **Macro-F1** — the mean of the per-section F1 scores, so every section counts equally — plus accuracy, on a held-out test set used once.

3.2 Dataset: collected by the group

No ready-made dataset was used: the group wrote a crawler — `crawl_vnexpress.py` in `text/crawler/`, built on `requests` and `BeautifulSoup` — for *VnExpress* [1], one of Vietnam’s largest online newspapers, and ran it on 27 September 2026.

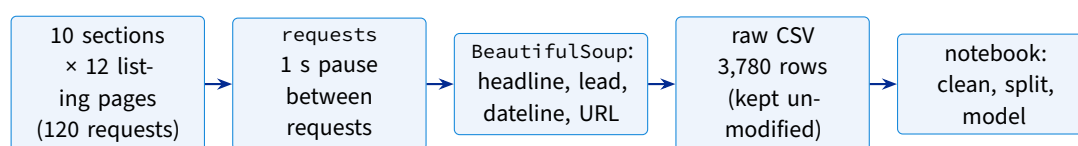


Figure 3.1. Data collection pipeline. Only headlines, leads and URLs are stored (no article bodies), for non-commercial coursework.

Table 3.1. Fields of the crawled dataset.

Column	Meaning
title	headline
description	lead paragraph (sapo)
location	dateline such as “Hà Nội” or “Indonesia”, when present (not used as a feature)
url	article URL — unique key
category_slug, category	section = label
page, crawled_at	listing page number and crawl time

Table 3.2. Assignment requirements for text data (section 4.3) and how this dataset meets them.

Requirement	This dataset	Status
At least 2,000 samples	3,780 articles (3,769 after cleaning)	Met
Vietnamese data encouraged	all headlines and leads are Vietnamese	Met
Self-collected data valued	crawled by the group; the crawler is in the repository	Met

3.3 Exploratory data analysis

Quality. 24 articles have no lead paragraph (mostly photo or video stories); 7 URLs appear twice within the same section (an article was pinned while the listing shifted between requests); 2 URLs are listed under two different sections, which makes their label ambiguous. 0 headlines needed Unicode normalisation.

Balance and length. The crawler read the same number of pages per section, so the classes are balanced (374–391 articles each). Texts are short: a median headline has 12 syllables and a median lead 31 — about the length of one tweet. Vietnamese separates *syllables*, not words, with spaces (“bóng đá”, football, is two tokens), which shapes the choice of representation (Section 3.4).

Table 3.3. The eight n-grams most over-represented in each section (χ^2 test of each TF-IDF 1–2-gram, section vs. rest, training set).

Section	Top 8 discriminative n-grams (χ^2 , training set)
Business	lãi, phiếu, doanh nghiệp, cổ phiếu, lãi suất, doanh, khoán, chứng khoán
Education	học, đại học, sinh, trường, học sinh, sinh viên, đại, ngành
Entertainment	phim, miss, diễn viên, miss world, diễn, hoa hậu, ca, hậu
Health	bệnh, huyết, bác sĩ, máu, viêm, ăn, uống, ung
Law	bị cáo, cáo, bị, cáo buộc, công an, sát hại, tù, lừa
News	ngập, mưa, mưa lớn, sông, quốc lộ, lộ num, sạt, lở
Sci-Tech	iphone, robot, num pro, pro, iphone num, apple, ai, robot hình
Sports	cup, hlv, asiad, man, trận, asiad num, league, fifa
Travel	khách, du, du khách, du lịch, điểm đến, khách việt, lịch, dịp
World	trump, iran, ukraine, ông trump, tổng thống, tổng, nga, houthi

Question. Which words does each section “own”?

Finding. Two kinds of signal appear. **Stable topic vocabulary:** *Health* — *bệnh, bác sĩ, viêm*; *Education* — *học sinh, đại học, sinh viên*; *Law* — *bị cáo, cáo buộc, tù*; *Business* — *cổ phiếu, lãi suất, chứng khoán*. Bigrams such as *cổ phiếu* and *bác sĩ* rank next to their single syllables, so syllable pairs recover real words. **Time-specific names:** *Sci-Tech* is driven by *iphone, apple, robot*, *World* by *trump, iran, ukraine*, *Sports* by *asiad, fifa*, and *News* by recent floods (*ngập, mưa lớn, sạt lở*). These reflect the news cycle at crawl time rather than the section itself — a risk discussed in the limitations.

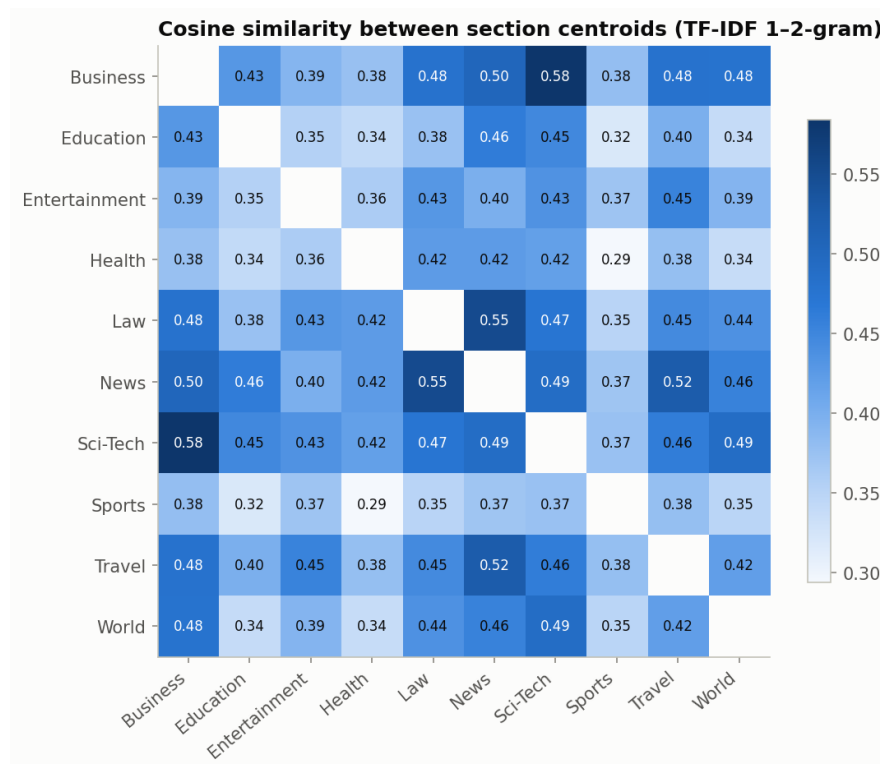


Figure 3.2. Cosine similarity between the average TF-IDF vector (centroid) of each section.

Question. Before training any model, can we predict which sections will be confused?

Finding. The most similar pairs are **Business and Sci-Tech** (0.58) — chips, technology companies and market value appear in both — and **Law and News** (0.55) — police and accidents appear in both. *Sports* is the most isolated section. Section 3.7 checks this prediction against the confusion matrix.

3.4 Text preparation

Table 3.4. Preparation steps.

Step	What is done and why
Duplicates	keep one copy of URLs repeated within a section; drop URLs listed under two sections (ambiguous label) → 3,769 articles (11 rows removed)
Missing values	a missing lead becomes an empty string, so the article is classified from its headline alone
Input text	headline + “ . ” + lead
Cleaning (VietnameseTextCleaner)	Unicode NFC normalisation (the same diacritic can be stored as one or two code points); lower-casing; numbers → token num; punctuation removed
Tokens	pattern <code>\b\w+\b</code> keeps one-character syllables; the scikit-learn default would silently drop 27 of them, including <code>ô</code> (in <code>ô tô</code> , <code>car</code>) and <code>ở</code> (<code>at</code>)
Label encoding	<code>LabelEncoder</code> : 10 sections → 0–9
Split	stratified 70 / 15 / 15: 2,638 / 565 / 566 articles

For text, *encoding* means turning each document into a vector and *scaling* means weighting and normalising that vector. Four representations are compared, all fitted on the training set only:

Table 3.5. Text representations compared.

Representation	Idea	Weighting / scaling	Vocabulary
BoW, 1-gram	how often each syllable occurs	raw counts	2,950
TF-IDF, 1-gram	counts re-weighted by rarity across documents	sublinear tf, L2 norm	2,950
TF-IDF, 1–2-gram	adds syllable pairs → compound words	sublinear tf, L2 norm	15,517
TF-IDF, char 2–5	character n-grams inside words → robust to spelling	sublinear tf, L2 norm	19,160

3.5 Modelling

3.5.1 Representation × classifier grid

Each representation is paired with four classifiers — Multinomial Naive Bayes, Logistic Regression, a linear Support Vector Machine and k-nearest neighbours with cosine distance — and scored on the validation set (TextBenchmark class).

Table 3.6. Validation macro-F1 for every representation × classifier pair (best in bold). A stratified random guess scores 0.081.

Representation	Multinomial NB	Logistic Reg.	Linear SVM	kNN (cosine)
BoW 1-gram	0.839	0.820	0.819	0.751
TF-IDF 1-gram	0.837	0.848	0.858	0.837
TF-IDF 1–2-gram	0.856	0.864	0.863	0.849
TF-IDF char 2–5	0.825	0.847	0.858	0.819

Question. Which representation and classifier work best for short Vietnamese texts?

Finding. **TF-IDF beats raw counts** for the discriminative models — Logistic Regression +0.028, Linear SVM +0.039, kNN +0.086 — because IDF down-weights syllables that occur everywhere (*của, là, và*), acting as an automatic stop-word list; Naive Bayes, designed for counts, does not gain. **Adding bigrams helps every model** (+0.005 to +0.019), while character n-grams do not beat word bigrams. The best pair is TF-IDF 1–2-gram + Logistic Regression (0.864); the Linear SVM on the same features is practically tied (0.863) and trains several times faster.

3.5.2 Tuning

The TF-IDF + Linear SVM pipeline was tuned with 5-fold stratified cross-validation on the training set over the n-gram range, `min_df`, sublinear term frequency and the regularisation constant `c`. The best setting uses 1–3-grams, `min_df` = 1 and `c` = 3 (cross-validated macro-F1 0.865). On the single validation split it scores 0.857 — within about 0.01 of the untuned models, i.e. a handful of the 565 articles; the 5-fold estimate is the more reliable of the two.

3.6 Extra experiments

3.6.1 Which field carries the signal, and would more data help?

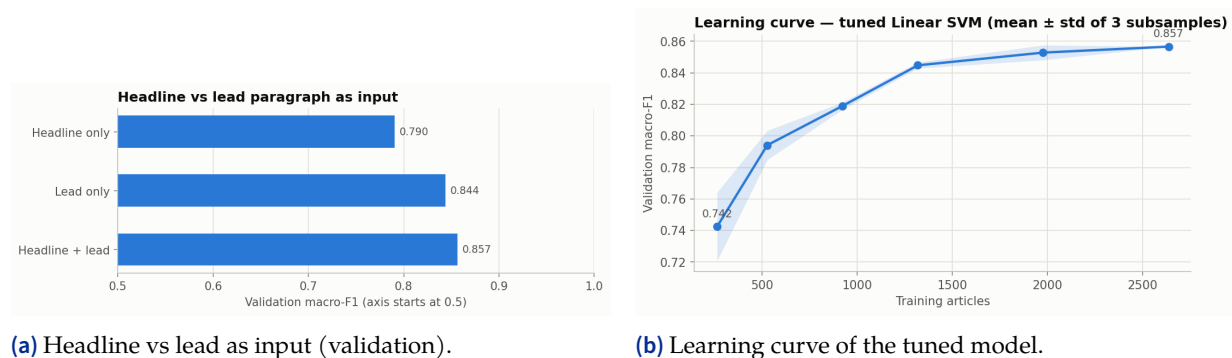


Figure 3.3. Two questions about the data itself, answered with the tuned TF-IDF + Linear SVM pipeline.

Question. Is the headline or the lead more informative — and is our crawled dataset big enough?

Finding. The **lead alone (0.844) beats the headline alone (0.790)** by +0.053: leads are longer and written to summarise, headlines are short and often figurative. Using both is best (0.857, +0.013 over the lead). The learning curve rises from 0.742 with 263 articles to 0.845 with 1,319, then flattens: doubling to 2,638 articles adds only +0.012. Crawling more would help a little; a better representation should help more.

3.6.2 TF-IDF versus multilingual sentence embeddings

We encode every article with `intfloat/multilingual-e5-small` [2], a 118-million-parameter multilingual sentence encoder with 384-dimensional output, loaded with the `sentence-transformers` library [3]. Following the assignment, the pretrained model is used as a **frozen feature extractor**: a Logistic Regression is trained on the embeddings. A **hybrid** model concatenates the TF-IDF features with the embeddings and trains a Linear SVM.

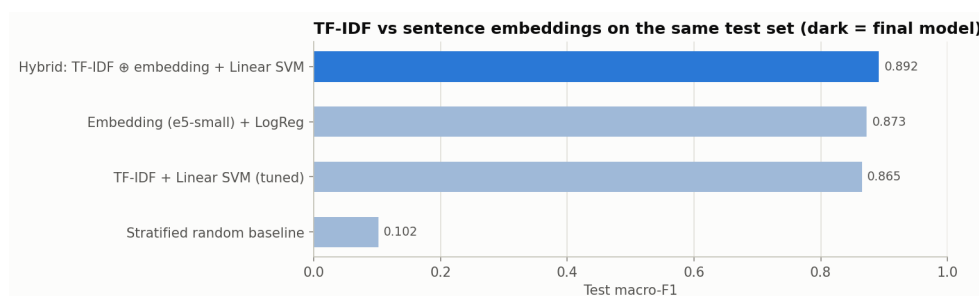


Figure 3.4. Test macro-F1 of the three approaches and the random baseline (dark = final model, chosen on validation).

Question. Does a pretrained neural encoder beat a bag of syllables on short Vietnamese news?

Finding. Yes, and combining them is best. On the test set, embeddings alone reach 0.873 (+0.007 over tuned TF-IDF at 0.865), and the **hybrid reaches 0.892** (+0.027). The ranking is the same on validation (0.857 → 0.882 → 0.888), so the choice generalises. TF-IDF keeps exact names (*iphone*, *asiad*) that embeddings blur; embeddings map different wordings of the same idea close together.

3.7 Results on the test set

Table 3.7. Validation and test scores of the compared approaches. The final model was chosen on validation macro-F1.

Approach	Val. macro-F1	Test macro-F1	Test acc.
Random guess (stratified)	—	0.102	—
TF-IDF 1–3-gram + Linear SVM (tuned)	0.857	0.865	0.866
Embedding (e5-small) + Logistic Regr.	0.882	0.873	0.873
Hybrid: TF-IDF ⊕ embedding + Linear SVM	0.888	0.892	0.892

The final hybrid model reaches **accuracy 0.892 and macro-F1 0.892** on the 566 test articles; 61 are misclassified. *Sports* is the easiest section (F1 0.973) and *Business* the hardest (0.814).

Table 3.8. Per-section test F1 of the final model.

Section	F1	Section	F1
Sports	0.973	Travel	0.878
Entertainment	0.946	News	0.865
Health	0.937	World	0.846
Education	0.924	Sci-Tech	0.844
Law	0.893	Business	0.814

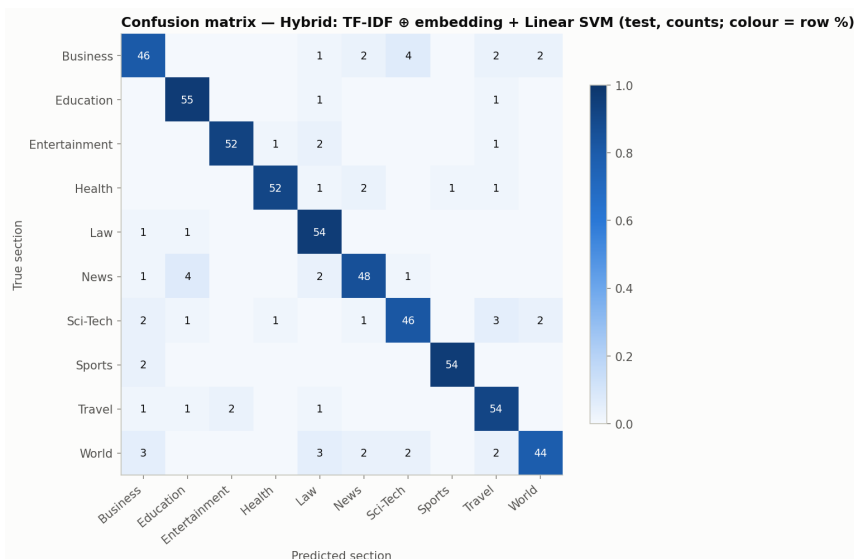


Figure 3.5. Confusion matrix of the final model on the test set (counts; colour = share of the true section).

Table 3.9. Most frequent confusions on the test set.

True section	Predicted as	Test errors
News	Education	4
Business	Sci-Tech	4
World	Law	3
Sci-Tech	Travel	3
World	Business	3

Question. Did the centroid similarity of Figure 3.2 predict the errors?

Finding. Largely yes: **Business** → **Sci-Tech**, the most similar pair before training, is among the two most frequent confusions. Business is the hardest section because it overlaps with technology, world affairs and travel (airlines, tourism revenue).

Table 3.10. Examples of misclassified test articles.

Headline (Vietnamese)	True	Predicted
Thủ đô quốc gia Đông Nam Á nào đông dân nhất thế giới?	Travel	Education
Giải cứu 10 ngư dân Việt Nam làm việc tại Malaysia	World	News
Hàng không đón khách quốc tế nhiều hơn nội địa dịp Quốc khánh	Business	Travel
Tại sao cửa sổ máy bay khó sản xuất?	Sci-Tech	Travel
Nghệ thuật 'khiêu khích' ẩn sau quả chuối dán tường 6 triệu USD	World	Travel
Nổi dậy dớt của đầu bếp thoát chết vụ 11/9 nhờ đổi ca với đồng nghiệp	World	Law
Justin Bieber mở show miễn phí vì người vô gia cư	Entertainment	Travel
Mark Zuckerberg mua lâu đài	Sci-Tech	Travel

Many errors sit genuinely between sections — “*Tại sao cửa sổ máy bay khó sản xuất?*” (why aircraft windows are hard to make) is Sci-Tech but reads like Travel; a human editor could file several of them either way, so part of the remaining error is label ambiguity rather than model failure. Finally, all eight headlines written by the group, which are not in the dataset, are classified correctly:

Table 3.11. Predictions of the final model on new headlines.

Headline written by the group (not in the dataset)	Predicted section
Đội tuyển Việt Nam thắng Thái Lan 2-1 ở chung kết	Sports
Giá vàng tăng mạnh phiên đầu tuần	Business
Bộ Giáo dục công bố phương án thi tốt nghiệp THPT 2027	Education
Phát hiện hành tinh mới có thể chứa nước	Sci-Tech
Bác sĩ khuyến cáo cách phòng sốt xuất huyết mùa mưa	Health
Khởi tố giám đốc công ty lừa đảo chiếm đoạt 50 tỷ đồng	Law
Tổng thống Mỹ gặp Chủ tịch Trung Quốc tại hội nghị APEC	World
Vịnh Hạ Long lọt top điểm đến đẹp nhất châu Á	Travel

3.8 Conclusions

1. **A self-crawled Vietnamese dataset is enough for a strong classifier:** 3,769 clean articles in 10 sections give 0.892 test macro-F1 from a headline and a two-sentence lead.
2. **Vietnamese needs care in the details:** NFC normalisation, keeping one-character syllables, and syllable bigrams to recover compound words. TF-IDF 1–2-grams beat raw unigram counts by +0.044 (Logistic Regression) and +0.044 (Linear SVM).
3. **Sentence embeddings beat TF-IDF, and the combination is best:** +0.007 and +0.027 test macro-F1 over tuned TF-IDF.
4. **The lead carries more signal than the headline** (0.844 vs 0.790), and the learning curve flattens after about 1,319 articles.
5. **Errors are predictable:** the most similar pair of section centroids is among the most frequent confusions.

Limitations. All articles come from one short window (the latest 12 listing pages per section on 27 September 2026), and many discriminative words are event names (Trump, ASIAD, iPhone, floods), so accuracy will likely drop on news from another period. There is one newspaper and one editorial style, and section labels are sometimes ambiguous. Syllable n-grams only approximate words; a Vietnamese word segmenter was not used, to keep the notebook dependency-free.

Future work. Re-crawl in a later month and measure the drop (temporal robustness); crawl other newspapers to test cross-source generalisation; compare word-segmented TF-IDF; lightly fine-tune a Vietnamese language model on top of the frozen-encoder baseline.

References

- [1] VnExpress — Vietnamese online newspaper. <https://vnexpress.net> (section listings crawled 27 September 2026).
- [2] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei. *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672, 2024. Model: <https://huggingface.co/intfloat/multilingual-e5-small>
- [3] N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
- [4] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12 (2011), 2825–2830.
- [5] Group G3. *Notebook G3_Text_VnExpressNews.ipynb*. https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/text/G3_Text_VnExpressNews.ipynb
- [6] Group G3. *Project page (rendered notebook, report and video)*. https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/text/ — video: to be published

Time-series data: forecasting Hanoi's daily temperature

4.1 Problem and objective

Hanoi has a real winter: cold surges of the north-east monsoon can cut the temperature by 5–10 °C within a day, and those are exactly the days when a forecast matters most for health, energy use and agriculture.

Objective. Forecast **tomorrow's daily mean temperature in Hanoi** from today's and past weather, then measure how quickly forecast skill decays when predicting up to seven days ahead.

How success is measured. Mean absolute error (MAE, °C) on a chronologically held-out test period, and *skill* relative to simple baselines — above all *persistence* (“tomorrow = today”), which is hard to beat for one-day forecasts.

4.2 Dataset

The data was downloaded by the group from the **Open-Meteo Historical Weather API** [1] (free, no API key, CC BY 4.0), which serves the **ERA5 reanalysis** [2]: a physically consistent reconstruction of past weather on a 9–25 km grid, combining a weather model with millions of observations. The exact request is included in the notebook (`fetch_open_meteo`); *Run all* uses the saved CSV so results are reproducible.

- Location: central Hanoi (21.03 °N, 105.85 °E; nearest grid cell 21.05 °N, 105.90 °E, elevation 19 m).
- Period: 1 January 2005 – 26 September 2026, daily, local time (GMT+7): 7,939 consecutive days, 14 variables.

Table 4.1. Variables. The target is the daily mean temperature.

Variable	Unit	Meaning
temperature_2m_mean / _max / _min	°C	daily mean (target), maximum, minimum at 2 m
apparent_temperature_mean	°C	“feels-like” temperature
precipitation_sum, precipitation_hours	mm, h	daily rain and rainy hours
relative_humidity_2m_mean, dew_point_2m_mean	%, °C	moisture
pressure_msl_mean	hPa	mean sea-level pressure
cloud_cover_mean	%	cloud cover
wind_speed_10m_max, wind_direction_10m_dominant	km/h, °	maximum wind speed, dominant direction
shortwave_radiation_sum, sunshine_duration	MJ/m ² , s	solar energy, sunshine

Table 4.2. General requirements (section 4.1 and 5) and how this dataset meets them.

Requirement	This dataset	Status
Enough samples	7,939 days	Met
Numerical columns (scaling)	14 weather variables plus engineered lags	Met
Categorical columns (encoding)	derived: season and 8-sector wind direction (one-hot), circular encodings	Met
Missing values handled	none in the source (verified); gaps simulated to compare imputation methods (Section 4.4.1)	Met
Time-aware split	train / validation / test by date; time-series cross-validation	Met

4.3 Exploratory data analysis

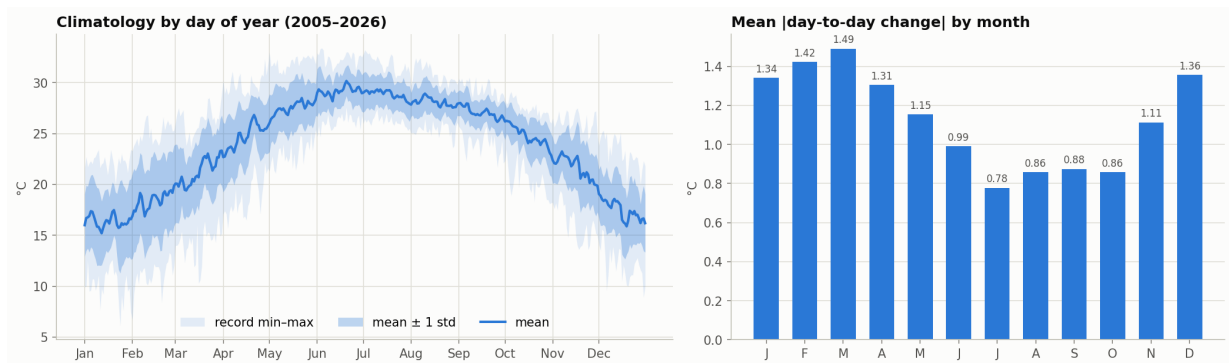


Figure 4.1. Left: climatology by day of year (mean, ± 1 standard deviation, record range). Right: mean absolute day-to-day change by month.

Question. How different are the seasons — and when is the weather hardest to predict?

Finding. Summer (June–August) averages **28.7 °C** and winter (December–February) **17.4 °C**, a swing of about 11 °C. Winter is also far more *variable*: the standard deviation within January is 3.40 °C versus 1.62 °C in July, and the average day-to-day change peaks in February–March (1.42–1.49 °C) against 0.78 °C in July. Forecasting should therefore be hardest in winter and spring.

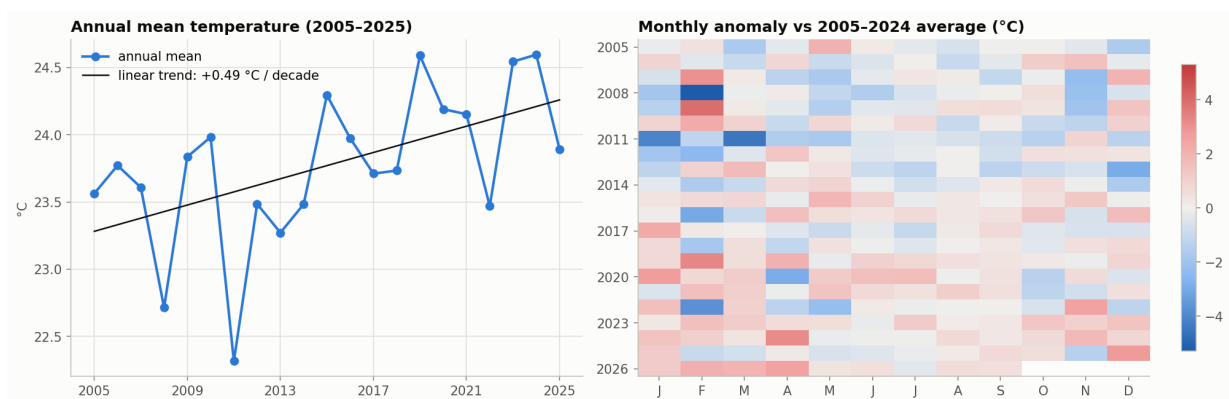
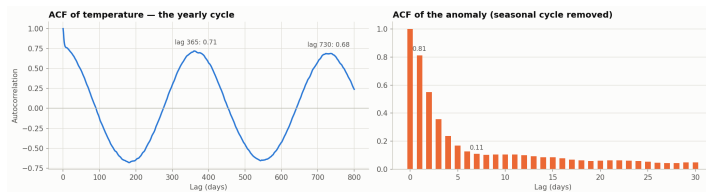


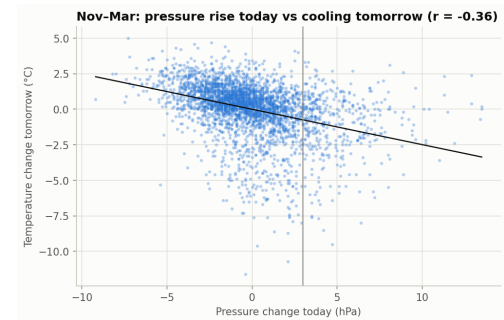
Figure 4.2. Left: annual mean temperature with a linear trend. Right: monthly anomaly versus the 2005–2024 average (blue = colder, red = warmer).

Question. Is Hanoi getting warmer?

Finding. The annual mean rises by **+0.49 °C per decade** over 2005–2025; 2021–25 averaged 24.13 °C against 23.50 °C in 2005–09 (+0.63 °C). Large cold anomalies cluster in the early years (February 2008, January–February 2011) and recent years are mostly warm. Twenty-one years is short and year-to-year swings are large (22.3 °C in 2011 vs 24.6 °C in 2024), so the slope is uncertain — but it matters for modelling: a climatology learned on 2005–2021 will be slightly too cold for 2024–2026 (Section 4.6).



(a) Autocorrelation of temperature and of its anomaly.



(b) Pressure change today vs temperature change tomorrow (Nov–Mar).

Figure 4.3. Memory of the series and the cold-surge signal.

Question. How much does today tell us about tomorrow, and can we see a cold surge coming?

Finding. The raw series is dominated by the yearly cycle (autocorrelation 0.71 at lag 365). Once the season is removed, the anomaly still has strong day-to-day memory (**0.81 at lag 1**) that fades to **0.11 by lag 7**: a warm or cold spell “remembers” itself for only a few days. In the cold season a pressure rise today warns of cooling tomorrow ($r = -0.36$): on the 404 days when pressure rose by more than 3 hPa, the next day changed by $-1.03\text{ }^{\circ}\text{C}$ on average (vs $+0.15\text{ }^{\circ}\text{C}$ otherwise) and a drop of more than $2\text{ }^{\circ}\text{C}$ was **2.7× as likely** (26.5% vs 9.6%).

4.4 Data preparation

4.4.1 Continuity and missing values

A time series must be checked for missing *dates*, not only missing cells, because a skipped day silently shifts every lag feature. The file has 7,939 rows for 7,939 calendar days, no duplicated dates and 0 missing cells. Real station data is rarely this clean, so we used the complete series as ground truth for an experiment: hide random blocks of 1–30 consecutive days (about 5% of the series, 5 repetitions) and reconstruct them with four methods.

Table 4.3. Imputation error (MAE in °C on the hidden days; best per gap length in bold).

Gap length	Forward fill	Linear interp.	Climatology	Seasonal interp.
1 day	1.16	0.71	1.97	0.70
3 days	1.76	1.14	1.89	1.13
7 days	2.21	1.69	2.00	1.63
14 days	2.45	1.95	1.86	1.93
30 days	2.80	2.22	1.87	2.15

Question. If a station lost data for a few days, how should the gap be filled — and does the answer depend on its length?

Finding. For short gaps, **seasonal interpolation** (interpolating the anomaly, then adding the climatology back) is best: 0.70, 1.13 and 1.63 °C for 1, 3 and 7 days, slightly ahead of plain linear interpolation (0.71, 1.14, 1.69). Forward fill is worst at every length. For gaps of **14 days or more, the seasonal normal alone wins** (1.86 °C), because the anomaly’s memory (about a week) has run out. Rule: interpolate the anomaly up to about a week, fall back to climatology for longer outages.

4.4.2 Outliers

Temperature outliers must be judged against the season (12 °C is normal in January but extreme in July), so each day’s anomaly is divided by the standard deviation for that day of the year and $|z| > 3$ is flagged. Only 9 days qualify, all cold (March 2011, early October 2011, September 2013): real cold spells, exactly what a forecast must handle, so they are **kept**. The record 6.1 °C of 24 January 2016 is not even flagged because January is so variable. Precipitation has 1,001 IQR outliers and skewness 5.2 (maximum 227 mm on 21 June 2010); heavy rain is also real, so it is kept and fed to the models as $\log_{1p}(\text{rain})$ (skewness 0.95).

4.4.3 Features, encoding and scaling

For a forecast issued at the end of day t for day $t + h$, every feature must be known on day t . The class `WeatherFeatureBuilder` builds them; its climatology is learned from the **training years only**.

Table 4.4. Features (30 in total) and their encoding.

Group	Features	Encoding / scaling
Recent temperature	today; lags 1, 2, 3, 7, 14 days; rolling means over 3, 7, 30 days; today's max, min and range	standardised
Anomaly	today – climatology (smoothed day-of-year mean)	standardised
Weather today	humidity, dew point, pressure, pressure change over 1 and 3 days, cloud cover, wind speed, rain, radiation, sunshine	log1p for rain; standardised
Wind direction	sine and cosine of the direction	circular (359° is next to 1°)
Calendar of target day	day of year; climatology of the target day	sine/cosine (cyclical)
Categorical	season of target day (DJF, MAM, JJA, SON); 8-sector wind direction (N, NE, ..., NW)	one-hot

The first 30 days (rolling-window warm-up) and the last day (no target) are dropped. Scaling uses `StandardScaler` inside the pipeline; it is required by Ridge regression and harmless for tree models.

4.4.4 Time-based split

A random split would let the model learn from days *after* the ones it is tested on. We split by the date being forecast: **train** 2005–2021 (6,179 days), **validation** 2022–2023 (730 days) and **test** 1 January 2024 – 26 September 2026 (1,000 days). Hyperparameters are tuned with `TimeSeriesSplit` (5 expanding windows inside the training years), which always trains on the past and validates on the following period.

4.5 Modelling

Baselines. *Persistence* (tomorrow = today); *climatology* (the average for that calendar day); and *damped anomaly persistence* (climatology + $\varphi \times$ today's anomaly, with $\varphi = 0.81$, the lag-1 autocorrelation of anomalies in the training years).

Machine-learning models. Ridge regression, Random Forest and HistGradientBoosting (HGB), all in the same pipeline. HGB is trained either on the temperature itself or on the **change** $\Delta T = T(t+1) - T(t)$, adding today's value back afterwards (`DeltaRegressor` class). Trees cannot extrapolate beyond the range seen in training, and a warming climate makes that a real risk; predicting the change avoids it.

Tuning. A random search over 20 HGB configurations (scikit-learn defaults included) with time-series cross-validation preferred a more regularised model (CV MAE 0.928 vs 0.944 for the defaults), but it did *worse* on 2022–2023 (0.922 vs 0.906). A 0.015 °C gain on older folds is within year-to-year noise, so the final model is the one chosen on validation: HGB on the change, with default hyperparameters.

Table 4.5. One-day-ahead forecasts: validation (2022–2023) and test (2024 – September 2026) scores. Skill = $1 - \text{MAE} / \text{MAE of persistence (test)}$.

Model	Val MAE	Test MAE	Test RMSE	Test R ²	Skill
Climatology (seasonal normal)	2.014	1.990	2.538	0.722	−84.2%
Persistence (tomorrow = today)	1.105	1.081	1.504	0.902	0.0%
Damped anomaly persistence	1.092	1.075	1.434	0.911	0.5%
Ridge regression	0.974	0.970	1.283	0.929	10.2%
Random Forest	0.921	0.935	1.245	0.933	13.4%
HGB, temperature as target	0.933	0.932	1.231	0.934	13.8%
HGB, change as target, tuned	0.922	0.919	1.222	0.935	15.0%
HGB, change as target (final)	0.906	0.916	1.226	0.935	15.3%

Question. Do machine-learning models beat the classical baselines, and which framing works best?

Finding. Every model beats every baseline. HGB predicting the change is best on validation (0.906 °C, 18% below persistence at 1.105) and beats HGB predicting the level (0.933). Ridge regression is weakest (0.974): the useful signals interact non-linearly — a northerly wind matters mostly in winter. Damped anomaly persistence barely improves on persistence (1.092) and climatology alone is poor (2.014 °C): for *tomorrow*, today's weather matters far more than the calendar.

4.6 Results on the test set

On the 1,000 unseen days the final model reaches **MAE 0.916 °C, RMSE 1.226 °C and R^2 0.935**, a **15.3% lower error than persistence** (1.081 °C); the ranking of all models matches validation (Table 4.5).

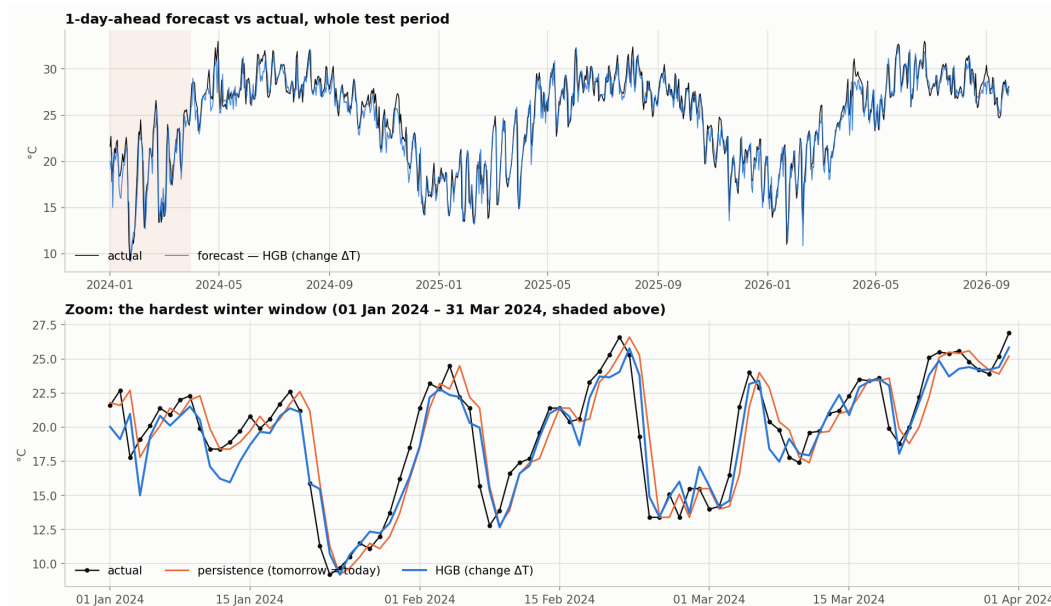


Figure 4.4. Forecast versus actual over the whole test period (top) and the hardest winter window (bottom), with persistence for comparison.

Table 4.6. Test MAE by month (bold = final model better than persistence).

Test MAE (°C)	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Persistence	1.36	1.61	1.24	1.08	1.08	0.99	0.75	0.87	0.86	0.96	1.01	1.12
Final model	1.05	1.16	0.82	1.10	0.91	0.87	0.69	0.84	0.72	0.90	1.12	0.88

Table 4.7. The 8 largest test errors (°C; pressure change on the forecast day in hPa).

Date	Actual	Forecast	Error	1-day change	Pressure change
29 March 2025	17.0	23.7	+6.7	−8.5	+4.2
8 February 2026	15.0	20.7	+5.7	−7.0	+2.2
18 November 2025	17.8	22.7	+4.9	−4.9	−0.1
23 February 2024	19.3	23.8	+4.5	−6.0	+3.7
26 January 2025	14.9	19.4	+4.5	−6.4	+1.6
29 April 2024	31.7	27.4	−4.3	+1.0	+2.3
7 February 2024	15.7	20.0	+4.3	−5.7	+0.8
22 January 2024	11.3	15.5	+4.2	−4.6	+3.5

Question. When does the model fail?

Finding. The biggest gains over persistence come in winter and spring — February 1.16 vs 1.61, March 0.82 vs 1.24, December 0.88 vs 1.12 °C — while the model is slightly worse in the transition months (April 1.10 vs 1.08, November 1.12 vs 1.01). **7 of the 8 worst forecasts are cold surges:** the temperature fell 4.6–8.5 °C in one day and the forecast was 4.2–6.7 °C too warm, usually after a pressure rise that the model reacted to only partly. On average the residual is -0.27 °C (standard deviation 1.20): forecasts are slightly too cold, consistent with the warming of Figure 4.2.

4.7 Extra experiments

4.7.1 How far ahead is the model useful?

The features are rebuilt and the final configuration retrained for horizons of 1 to 7 days; all methods are scored on the same test period.

Table 4.8. Test MAE (°C) by forecast horizon (best per horizon in bold).

Test MAE (°C)	1 d	2 d	3 d	4 d	5 d	6 d	7 d
Persistence (tomorrow = today)	1.08	1.74	2.16	2.44	2.56	2.60	2.61
Climatology (seasonal normal)	1.99	1.99	1.99	1.99	1.99	1.99	1.99
Damped anomaly persistence	1.07	1.60	1.86	1.97	2.00	1.99	1.99
Final model	0.92	1.49	1.79	1.95	1.99	2.00	1.99

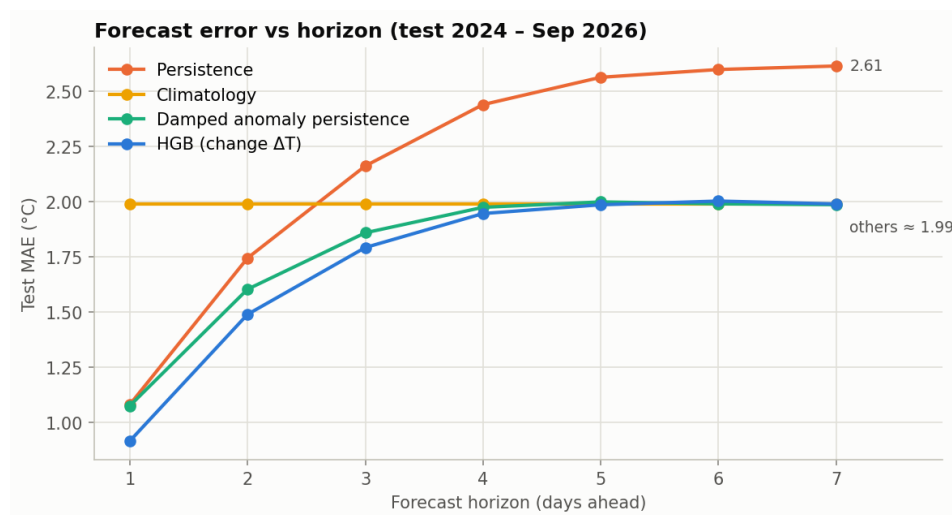


Figure 4.5. Forecast error versus horizon on the test period.

Question. At what horizon does the model stop beating simple baselines?

Finding. The model beats every baseline for **1 to 4 days ahead** (MAE 0.916, 1.489, 1.792, 1.946 °C vs 1.075, 1.603, 1.858, 1.974 for the best baseline). From day 5 all methods converge on climatology (about 1.990 °C): the anomaly memory has faded, exactly as the autocorrelation of Figure 4.3 predicted. Persistence becomes worse than the seasonal average from day 3 (2.161 °C).

4.7.2 What drives the predicted change?

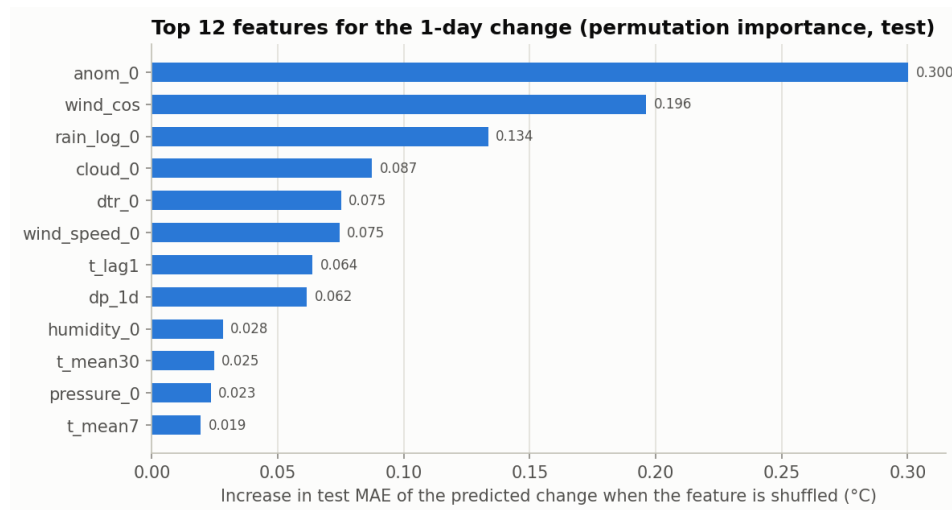


Figure 4.6. Permutation importance of each feature for the predicted one-day change (test set).

Question. Which signals does the model use to predict tomorrow's change?

Finding. Today's **anomaly** (0.300) comes first — an unusually warm day tends to cool back towards normal and vice versa. Next is the **north–south component of the wind** (0.196): a northerly wind brings cold air, the signature of a cold surge. Rain (0.134) and cloud cover (0.087) follow, as they suppress daytime heating. The one-day pressure change is used (0.062) but ranks lower than Figure 4.3 suggests, because the wind direction carries much of the same information.

4.8 Conclusions

1. **Machine learning beats the classical baselines, modestly:** 15.3% lower test MAE than persistence (0.916 vs 1.081 °C). Framing the target as the *change* helped more than hyperparameter tuning.
2. **The hard days are cold surges.** Winter and spring are where the model gains most and where its worst misses happen; the physical signals — northerly wind, rising pressure, today's anomaly — are what it learned to use.
3. **Skill fades after about 4 days**, when every method converges on the seasonal average, matching the roughly one-week memory of temperature anomalies.
4. **Data preparation lessons:** split by time, never randomly; fit the climatology on training years only; fill short gaps by interpolating the anomaly and long gaps with climatology.
5. **Hanoi is warming** by about +0.49 °C per decade in this record — enough to leave a −0.27 °C bias in forecasts trained on earlier years.

Limitations. ERA5 is a smoothed grid-cell value rather than a thermometer in the city; real station data would be noisier. The forecast uses the complete weather of day t , slightly more than an evening forecast would have. Only local variables are used, while cold surges start hundreds of kilometres to the north.

Future work. Add upstream features (pressure and temperature in southern China a day earlier) to see cold surges coming; use hourly data for 24-hour forecasts; produce probabilistic forecasts (quantile regression) to warn of large drops; compare with archived forecasts from a physics-based weather model; retrain regularly to follow the warming trend.

References

- [1] Open-Meteo. *Historical Weather API* (weather data by Open-Meteo.com, CC BY 4.0). <https://open-meteo.com/en/docs/historical-weather-api>
- [2] H. Hersbach et al. *The ERA5 global reanalysis*. Quarterly Journal of the Royal Meteorological Society, 146 (730), 2020, 1999–2049. <https://doi.org/10.1002/qj.3803>
- [3] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12 (2011), 2825–2830.
- [4] Group G3. *Notebook G3_TimeSeries_HanoiWeather.ipynb*. https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/timeseries/G3_TimeSeries_HanoiWeather.ipynb
- [5] Group G3. *Project page (rendered notebook, report and video)*. https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/timeseries/ — video: to be published

Overall conclusions

5.1 What the extra experiments showed

Table 5.1. One question per extra experiment, with its numerical answer.

Part	Question	Answer (numbers from the chapters above)
Tabular	How much do leakage and duplicates inflate the scores?	leakage: 1.000 on every metric; duplicates: F1 0.793 instead of 0.653
Tabular	Does handling the class imbalance help?	recall 0.613 → 0.784 at threshold 0.33; PR-AUC unchanged (0.768)
Text	Does a pretrained encoder beat TF-IDF?	test macro-F1 0.865 (TF-IDF) → 0.873 (embedding) → 0.892 (hybrid)
Text	Is the headline or the lead more informative?	lead 0.844 vs headline 0.790; both 0.857
Time series	How should missing days be filled?	seasonal interpolation up to 7 days (1.63 °C); climatology from 14 days (1.86 °C)
Time series	How far ahead is the forecast useful?	beats every baseline for 1–4 days; converges to climatology (1.990 °C) after

5.2 Lessons across the three parts

1. **Data preparation decides whether a result can be trusted.** Leakage columns and duplicated rows in the tabular data, ambiguous labels in the crawled news, and a random instead of chronological split in the time series would each have produced better-looking but false numbers. In every part the honest set-up is documented and, where possible, measured.
2. **Always compare with a baseline.** A dummy classifier (accuracy 0.724 but F1 0), a random guess (macro-F1 0.102) and persistence (1.081 °C) put every score in context; the gains of the final models are real but measured against these, not against zero.
3. **Framing matters as much as the algorithm.** Choosing the decision threshold (tabular), combining two representations (text) and predicting the change instead of the level (time series) each helped more than hyperparameter tuning did.
4. **Errors are explainable.** Online-travel-agency bookings, overlapping news sections and winter cold surges account for most mistakes, and each was anticipated by the exploratory analysis.

5.3 Limitations and next steps

The tabular model was evaluated with a random split and would need a chronological test before deployment; the news dataset covers one newspaper and one short period; the weather model uses only local variables and a reanalysis grid cell. The most promising next steps are a time-based evaluation for the hotel data, a second crawl a few months later to measure how the text model ages, and upstream weather features to anticipate cold surges earlier.

Links and reproduction

Table A.1. Full addresses of every deliverable.

Item	Address
Website (landing page)	https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/
Source repository	https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment
Tabular — page	https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/tabular/
Tabular — notebook	https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/tabular/G3_Tabular_HotelBooking.ipynb
Tabular — video	to be published
Text — page	https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/text/
Text — notebook	https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/text/G3_Text_VnExpressNews.ipynb
Text — video	to be published
Time series — page	https://trungly-hcmut.github.io/C05177_DataVisualization_Assignment/timeseries/
Time series — notebook	https://github.com/trungly-hcmut/C05177_DataVisualization_Assignment/blob/main/timeseries/G3_TimeSeries_HanoiWeather.ipynb
Time series — video	to be published

Re-running the analyses. Open a notebook from the repository in Google Colab (*File* → *Open notebook* → *GitHub*) and choose *Runtime* → *Run all*. Each notebook loads its data from the repository and needs only the libraries pre-installed on Colab; the text notebook additionally downloads the `multilingual-e5-small` model on first run.

Rebuilding the website and reports (from the repository root):

- `python tools/build_pages.py` — regenerates the three project pages from the executed notebooks;
- `python tools/build_reports.py` — regenerates these PDF reports (XeLaTeX) from the notebooks' saved results.